

Clinical Memory AI — Project Guide

A study guide for understanding and defending this project.

Looking for the condensed version to revise from? `INTERVIEW_PREP.md` (13 pages) has the pitches, the numbers, the five stories and the twenty-five most likely questions. This document is the full reference behind it.

This document explains what the system does, how every part works, what is genuinely built here versus borrowed, and how to talk about all of it. Each section starts in plain language and then goes technical.

Contents

1. The problem
2. Why clinical documentation is hard
3. End-to-end workflow
4. Frontend architecture
5. Backend architecture
6. Database, PostgreSQL and Supabase
7. Row-Level Security and multi-tenancy
8. Authentication and authorisation
9. The consultation workflow in code
10. Speech-to-text and code-mixed language
11. LLM structuring and provider failover
12. Clinical facts and longitudinal memory
13. Decision support and evidence grounding
14. Citation verification
15. The ML: escalation-risk model
16. Data analytics: longitudinal statistics
17. Evaluation
18. Security
19. Audit trail and attestation
20. Observability
21. Testing and CI
22. Built here vs external — and what not to claim
23. Interview preparation
24. Interview questions and answers

1. The problem

Plainly. A doctor in a busy Indian clinic sees a patient for eight to twelve minutes. In that time they must listen, examine, remember what happened at the last visit, decide what tests to order, choose a medicine, and write it all down. Something gives, and it is usually the writing down. Notes end up as three words. When the patient returns in six weeks — possibly to a different doctor in the same clinic — the record does not say enough to reconstruct what happened.

The technical framing. This is a data-capture problem with three constraints that shape the whole design:

- **The input is speech**, and it is code-mixed Hindi and English.
- **The output is a legal document.** A clinical note is evidence. It has to say who wrote it, when, and on what basis.
- **The system must not be trusted.** AI transcription and AI structuring both make errors, and some of those errors are clinically dangerous. So every AI output must be a *proposal* that a physician converts into a *fact*.

That third constraint is the one that produced most of the architecture.

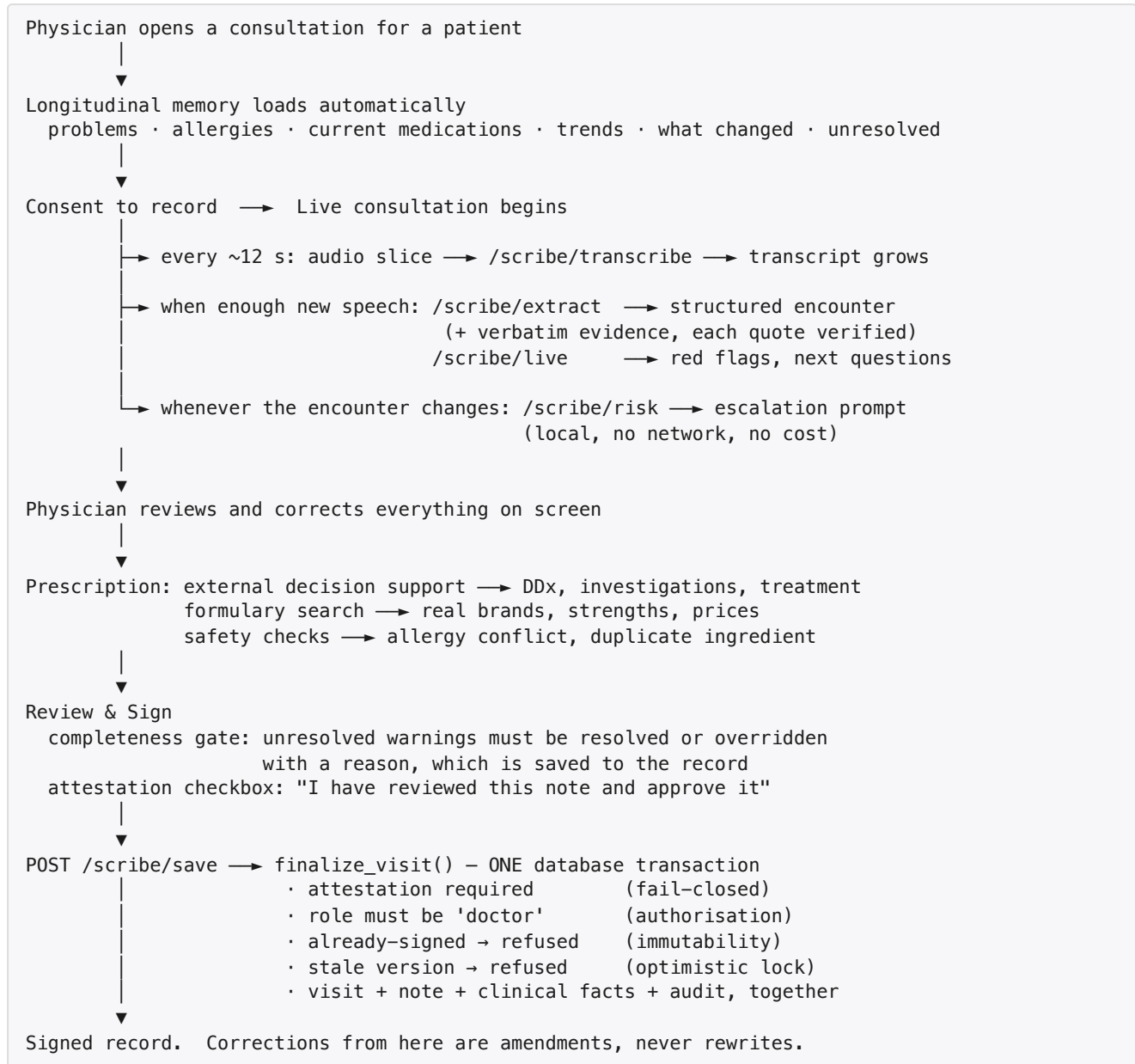
2. Why clinical documentation is hard

Plainly. It is not just typing. A good note has to record what the patient said, what the doctor found, what they concluded, and what they plan — in a way another clinician can act on months later. And it has to be true.

The specific difficulties this system deals with:

Difficulty	How it shows up
Speech is messy	Fillers, restarts, two languages in one sentence, background noise
Drug names are hard to hear	“Amlodipine” and “amlodipine 10” sound similar; a dose error is a patient-safety event
Absence is information	“No known drug allergies” is a <i>complete</i> allergy history. A blank field is not the same thing, and treating them alike is a real hazard
Provenance matters	“The AI said so” is not acceptable provenance in a medical record
History is scattered	The relevant fact might be from three visits ago
Legal weight	Once signed, the note is what the physician attested to. Rewriting it later destroys the record of what was decided at the time

3. End-to-end workflow



4. Frontend architecture

Plainly. A Next.js web app. The doctor's screen.

Technical.

- **Next.js 16, App Router, TypeScript, Tailwind.** Pages are client components because they need the microphone, timers and live state.
- **lib/api.ts** is the only place that talks to the backend. It attaches the Supabase session token to every request and translates failures into messages a clinician can act on. That translation matters: the backend distinguishes 409 (someone else edited this) from 403 (your role may not) from 429 (slow down), and collapsing them into "Save failed (409)" throws the distinction away at the moment the user needs it.

- **lib/supabaseClient.ts** creates the Supabase client **lazily**. Creating it at import time made `next build` fail on any checkout without credentials, because Next evaluates client-component modules during prerender. Building a project should not require production secrets.
- **lib/safety.ts** holds the prescription safety checks, extracted so they are unit-testable — and because the original inline version was wrong.
- **Client-side auth checks shape the UI only**. Every page checks for a session and redirects to `/login`, but that is convenience. The backend is the authority and re-derives identity from the token on every request.

Key screens: `/consult` (the wizard: consultation → prescription → review & sign), `/patients/[id]` (record + longitudinal analytics), `/consultations` (clinic dashboard), `/visits/[id]/print` (printable prescription).

5. Backend architecture

Plainly. A FastAPI service. Everything the browser cannot be trusted with.

app/	
├ main.py	app, middleware order, global error handler
├ core/	
│ └ config.py	Settings – required keys have no default, so a misconfigured server refuses to start
│ └ supabase.py	PostgREST + Auth access; pooled client; audit writes
│ └ pgrst.py	safe PostgREST filter construction
│ └ metrics.py	dependency-free Prometheus-format registry
│ └ ratelimit.py	bounded sliding-window limiter
│ └ budget.py	daily AI spend ceiling
│ └ observability.py	JSON logging, correlation ids, request metrics
├ ai/	
│ └ providers.py	LLM chain with cross-vendor runtime failover
│ └ stt.py	speech-to-text chain, same pattern
│ └ prompts.py	every prompt in one reviewable place
│ └ json_repair.py	structural repair of truncated model output
│ └ citations.py	quote verification against the transcript
├ clinical/	
│ └ completeness.py	deterministic documentation rubric
│ └ longitudinal.py	Mann-Kendall, Theil-Sen, step detection
│ └ risk.py	escalation prompt + deterministic criteria
├ ml/	
│ └ features.py	named clinical features
│ └ synthetic.py	the synthetic cohort generator
│ └ risk_model.py	pure-Python inference
│ └ risk_model.json	the exported artefact
├ api/	
│ └ deps.py	authentication, identity cache, role
│ └ routers/	health, me, clinics, patients, visits, scribe, drugs, match, conditions, synthesis

Middleware order is `CORS → RequestLog → RateLimit → app`, so CORS headers wrap every response including 429s, and the access log records rate-limited requests too.

Why FastAPI. The clinical payloads are deeply nested and come from a model, which means they are untrusted input. Pydantic validates them at the boundary, and the same models generate the OpenAPI schema the route-coverage test walks to prove no endpoint ships without authentication.

6. Database, PostgreSQL and Supabase

Plainly. PostgreSQL is the database. Supabase is a hosted PostgreSQL with authentication and an auto-generated REST API on top.

What Supabase gives us:

Piece	What it is
PostgreSQL	The database itself, with RLS
Auth	Sign-up, sign-in, JWT issuing and refresh
PostgREST	Turns tables and functions into a REST API, and — critically — sets <code>request.jwt.claims</code> per request so RLS policies see the caller

Why route through the backend rather than letting the browser call PostgREST directly? Three reasons: business rules (attestation, role, version checks) have to run somewhere the client cannot skip; provider keys must never reach the browser; and the audit trail must be written by something the user does not control.

Core tables

Table	Purpose
<code>clinics</code>	The tenant root
<code>users</code>	Maps a Supabase auth user to a clinic and a role
<code>patients</code>	Clinic-scoped, with a generated UHID
<code>visits</code>	One per consultation. Status, consent, <code>version</code> for locking
<code>soap_notes</code>	One per visit (unique index). The note, plus attestation and amendments
<code>clinical_facts</code>	Append-only. The longitudinal memory
<code>audit_log</code>	Append-only, hash-chained
<code>kb_*</code>	Global read-only reference data: conditions, terms, formulary

Why `clinical_facts` is separate from `soap_notes`. The note is a document; facts are *queryable assertions* with a type, a value, a source, a status, a temporal status, an author and a timestamp. You cannot compute a Mann-Kendall trend over prose. Splitting them is what turns “memory” from a list into something you can do statistics on.

7. Row-Level Security and multi-tenancy

Plainly. Several clinics share one database. Clinic A must never see clinic B's patients — even if a bug in our code asks for them.

How it works.

```
create function public.current_clinic_id() returns uuid
language sql stable security definer set search_path = public as $$
  select clinic_id from public.users where auth_uid = auth.uid() limit 1;
$$;

create policy patients_select on public.patients
  for select using (clinic_id = public.current_clinic_id());
create policy patients_update on public.patients
  for update using (clinic_id = public.current_clinic_id())
  with check (clinic_id = public.current_clinic_id());
```

`auth.uid()` reads the JWT claims PostgREST set for this request. The backend forwards the **caller's own token**, so the database evaluates the policy against their identity, not the server's.

USING vs WITH CHECK — the distinction people get wrong. **USING** decides which rows you can *see and target*. **WITH CHECK** validates the rows you are *writing*. Without **WITH CHECK**, an UPDATE could take a row you legitimately own and set `clinic_id` to another clinic — walking it out of the tenant. There is a test for exactly that.

Why this and not `where clinic_id = ...` in application code? Because that approach fails the day someone forgets one clause on one query. Here, no query in the codebase filters by clinic at all. Isolation does not depend on anyone remembering anything.

How it is proven. 31 database tests run the real migrations against a real PostgreSQL and then try to break out: read another clinic's patient by id, insert into another clinic, move a row between clinics, read another clinic's audit trail. Plus structural tests: RLS enabled on every public table, every writable clinical table has SELECT/INSERT/UPDATE policies, no policy is `using (true)`.

That last structural test exists because of a real bug: `soap_notes` had RLS on with SELECT, INSERT and DELETE policies but **no UPDATE policy**. PostgREST returns 204 for an update matching zero rows, so finalising a draft reported success and wrote nothing. Silent clinical data loss that looked exactly like success.

Honest limitation. `service_role` bypasses RLS entirely. It is used for exactly one thing — resolving `auth_uid → users` during authentication, before we know the clinic — and any new use should be treated as a review flag.

8. Authentication and authorisation

Plainly. Authentication is *who are you*. Authorisation is *what may you do*.

Authentication. The browser signs in with Supabase and gets a JWT. Every API call carries it.

`get_current_user` validates it against `/auth/v1/user` (rather than verifying the signature locally, so a revoked session stops working immediately and the code stays correct whether the project uses HMAC or

asymmetric keys), then looks up the user's clinic and role.

That is two network round trips per request, which dominates latency in a live consultation firing several calls a minute. So the resolved identity is cached for 30 seconds. The trade-off, stated: a session revoked at Supabase stays usable here for up to 30 seconds.

The identity always comes from the token, never from the request body. A client cannot assert which clinic it belongs to.

Authorisation. Two roles:

- `doctor` — may attest, amend, and remove records.
- `staff` — may prepare and save drafts, but may not sign.

That split mirrors how a clinic actually runs: the front desk prepares the consultation, the physician signs it. Attestation is a legal act, so the role check is on attestation rather than on writing.

Enforced in two places on purpose: the API returns a clear 403, and `finalize_visit()` re-checks it so it cannot be bypassed by anything holding a database session.

Error mapping that matters: an unreachable auth service returns **503**, not 401. Returning 401 during an outage sends every logged-in clinician to the login screen mid-consultation, where their retry fails the same way.

9. The consultation workflow in code

The heart of the system is `finalize_visit()` — a PL/pgSQL function, `SECURITY INVOKER` so RLS still applies (a `SECURITY DEFINER` version would be a tenant-isolation bypass).

```
select public.finalize_visit(
  p_patient_id      := '...',
  p_note            := '{...}'::jsonb,
  p_facts           := '[...]'::jsonb,
  p_visit_id        := '...', -- null to create
  p_expected_version := 7,     -- optimistic lock
  p_draft           := false,
  p_attested        := true,
  p_consent_given   := true,
  p_consent_method  := 'verbal'
);
```

In one transaction it:

1. Resolves the caller from their JWT. **There is no clinic parameter.**
2. Refuses to approve without `p_attested`.
3. Refuses to approve unless the role is `doctor`.
4. Checks the patient is visible (RLS makes this clinic-scoped automatically).
5. `SELECT ... FOR UPDATE` on the visit, so a concurrent finalize blocks here.
6. Refuses if already signed — a signed record is not rewritten.
7. Refuses if `p_expected_version` no longer matches.

8. Upserts the visit and the note.
9. Supersedes prior confirmed facts, then inserts the new ones — **drafts write no facts**, because an unsigned note must never become permanent history.
10. Writes the audit entry.

Why one function instead of five REST calls. The old flow was update visit, update note, supersede facts, insert facts, write audit. A failure part-way through left an approved visit with a stale note, or confirmed facts with no audit entry. There is now a test that injects an invalid fact type and asserts that nothing at all was written.

Optimistic locking, concretely. `GET /visits/{id}` returns `version`. The client sends it back on save. If someone else wrote in between, the version has advanced and the write is refused with `SQLSTATE 40001`, which the API maps to **409** and the UI renders as “This record was changed by someone else. Reload and try again.” Tested with two real connections racing.

10. Speech-to-text and code-mixed language

Plainly. Indian doctors do not speak one language in a consultation. A real sentence is: “*mujhe chest pain hai since two days, aur sweating ho rahi hai.*” Hindi grammar, English clinical vocabulary.

Why this is hard. Pin the recogniser to Hindi and it mangles the English clinical terms — which are the words that matter. Pin it to English and the Hindi disappears. So both providers are asked to **auto-detect** rather than being given a language.

The chain. OpenAI `gpt-4o-transcribe` first (strongest on code-mixed audio), Sarvam second (built for Indian languages). It fails over **at runtime**.

That distinction is the whole point. The previous implementation chose a provider at *configuration* time: if `OPENAI_API_KEY` was set it used OpenAI and, on failure, returned 502 — Sarvam was never tried, even sitting configured and healthy. In a live consultation that means the doctor’s speech is lost.

How the client streams. The browser captures raw PCM via `AudioContext`, encodes 16-bit mono WAV in JavaScript, and posts a slice roughly every 12 seconds. Only *new* audio is sent each time; the transcript is accumulated on the client. Twelve seconds is a compromise: shorter means more requests and more cost, longer means the note visibly lags the conversation.

Guards on the endpoint: a streaming read with a size ceiling (the previous unbounded `await file.read()` let one request pull an arbitrary payload into memory before any check ran), a content-type allowlist checked *before* the paid API is called, and a rejection of empty uploads.

11. LLM structuring and provider failover

Plainly. The transcript is turned into structured fields by a language model. Language models are unreliable, so the code treats them as unreliable.

The chain. `(gemini, gemini-2.5-flash) → (gemini, gemini-2.0-flash) → (gemini, gemini-2.5-flash-lite) → (openai, gpt-4o-mini)`.

Models within a vendor first, because provider-side model overload is the most common failure. Then a **different vendor** — a fallback sharing an outage with its primary is not a fallback.

What counts as a failure and what happens:

Failure	Behaviour
429, 5xx, timeout, connection error	Transient. Try the next candidate
401, 400	Skip the rest of <i>that vendor's</i> models — a bad key will not get better — but still fall through to the next vendor
200 with unparseable output	A failure, not a success. Fall over

That last row matters: treating a 200 carrying prose as success produced an empty note with no error anywhere.

JSON repair. Every provider supports a JSON mode and every one still occasionally returns fences, a preamble, or an object truncated at the token cap. `json_repair.py` handles exactly those three, with one rule: **repairing structure is allowed, inventing content is not**. A truncated field is dropped, never guessed. A dangling key disappears rather than acquiring a value. The brace matching is string-aware, because clinical free text contains braces (`"Tab Paracetamol {500mg} BD"`) and a naive counter corrupts real notes.

There is a property test that truncates a real note at every offset and asserts the repaired object is always a subset of the original with unchanged values.

Everything is measured — latency, outcome, tokens, estimated cost, and fallbacks — per capability, per provider. “How often does Gemini fail” is a number, not a feeling.

12. Clinical facts and longitudinal memory

Plainly. “Memory” means the system knows this patient’s story without anyone retyping it.

The model. Each fact is a row in `clinical_facts` :

Column	Meaning
fact_type	diagnosis, medication, allergy, symptom, vital, lab_result, follow_up
value	The text
structured	JSON — e.g. <code>{"context": "prescribed"}, {"metric": "bp", "reading": "138/86"}</code>
source	ai_extracted / doctor_entered / doctor_confirmed_ai
status	Provenance lifecycle: proposed → confirmed → superseded
clinical_status	Temporal meaning: current / historical / resolved / unknown
asserted_by, asserted_at	Who and when
valid_to	When it stopped being current

Two axes, and this is a point worth making in an interview. `status` is about *where the fact came from and whether a human accepted it*. `clinical_status` is about *whether it is true now*. A fact can be `confirmed` and `resolved` at the same time — the physician definitely signed it, and the condition has since gone away. Collapsing them means longitudinal memory reads every historical fact as present-tense, which is exactly the failure that makes an EMR's problem list untrustworthy.

Append-only, enforced. “Append-only” used to be a convention enforced by nothing. Now a trigger blocks any change to a fact's value, type, patient, visit or author, permits only forward status transitions, and `DELETE` is revoked.

Two distinctions that carry clinical weight:

- **Reported vs prescribed medications.** A medication the patient reported is history; one this clinic prescribed is the current regimen. Conflating them makes the medication timeline report stopping a drug that was never started.
- **Documented-negative allergies.** “No known drug allergies” is stored as an allergy fact with `structured.documented_negative = true` — not as an allergen named “No known drug allergies”. The UI shows three distinct states: *allergies listed*, *none known (documented)*, *not recorded*.

13. Decision support and evidence grounding

Be precise here — it is the thing most worth being honest about.

Ranked differential diagnoses, investigations by urgency, and treatment recommendations come from an **external Clinical Synthesis API this project did not build**. What is built here is the integration:

- A server-side proxy, because the upstream base URL is a shared secret and the service is unauthenticated — exposing it to the browser would hand anyone with devtools an open endpoint on someone else's bill.

- Three independent lanes fired in parallel (`asyncio.gather`), so a slow treatment lane does not delay the differential.
- Response normalisation, because upstream shapes drift.
- **Fail-open, loudly.** An upstream error returns empty lists with `available: false` rather than a 5xx, because an outage must not stop a physician documenting. But the client is required to show an explicit “unavailable” banner: the dangerous failure is a doctor reading an empty differential as *nothing to worry about*. Every failure is counted, so “fails open” cannot quietly become “always empty”.

A bug this surfaced. When decision support was unavailable, the consultation dead-ended: the only route from Prescription to Review & Sign went through “select a primary diagnosis”, and there was nothing to select. Losing suggestions made it impossible to sign a note. Found while writing a component test.

On RAG. There is no vector store and no embedding retrieval in this system, and adding one was considered and rejected. The two retrieval problems here are patient history (a bounded set of rows for one patient — SQL is the right tool) and clinical knowledge (owned by the external service). A vector database would be a keyword on a CV, not an improvement. The grounding that *does* exist — verifying every model quote against the transcript — is the part that actually reduces hallucination risk, and it is measured.

14. Citation verification

Plainly. When the system auto-fills “chest pain for two days”, it shows the words from the recording that produced it. And it checks they were really said.

Why. Asking a model to justify itself is worth nothing on its own — models produce fluent quotes that were never spoken. Verification is what turns the request into a guarantee.

Three verdicts:

Verdict	Rule	Counts as supported?
exact	Present after normalisation (case, unicode punctuation, whitespace)	Yes
near	A transcript window matches ≥ 0.82 by sequence similarity and contains every content word of the quote	Yes
unsupported	Anything else	No — dropped

The content-word requirement is the safety property. A model tidying “*uh, chest pain since, since two days*” into “*chest pain since two days*” is benign and should keep its evidence — reject it and the physician sees fields with no evidence and stops trusting the ones that have it. But “*headache since two days*” must fail, and it does: “headache” is not in the transcript, so no amount of surrounding similarity can pass it.

Measured, not asserted. On a labelled synthetic set: **precision 1.000, recall 1.000, specificity 1.000** (13 supported, 13 fabricated). Precision is gated at 1.00 in CI — accepting a fabricated quote is the one failure this system must not have, because a fabricated quote presented as verified evidence converts a physician’s scepticism into misplaced trust.

`evidence_coverage_pct` — the share of populated fields carrying a verified quote — is returned with every extraction and exported as a metric.

15. The ML: escalation-risk model

Full detail in the [model card](#). The essentials:

The question it answers. Not “what does this patient have”. Only: *does this case warrant a second look before the consultation ends?*

Features — 46, every one named and readable. Vital-sign cut-points from standard early-warning ranges (`spo2_lt_92` , `sbp_lt_90` , `rr_ge_25` ...), a symptom lexicon including Hinglish surface forms, comorbidity flags from history and medications, duration, complaint count, and documentation sparsity.

Negation is handled with a 30-character look-back, so “no chest pain” does not fire `sym_chest_pain` . Getting that wrong inverts the meaning of the input.

Model: L2 logistic regression, `class_weight="balanced"` , isotonic calibration. A gradient-boosted tree was trained as a reference and scored marginally better (ROC-AUC 0.909 vs 0.902). It is not deployed, because a physician has to be able to read *why* a case was raised and disagree with it — and two points of AUC on synthetic data does not buy an explanation nobody can audit in a two-minute consultation.

Results (held-out synthetic, n=1500):

Metric	Value
ROC-AUC	0.902
PR-AUC (baseline 0.201)	0.795
Precision @ 0.11	0.664
Recall / sensitivity	0.821
Specificity	0.896
F1	0.734
ECE	0.100 → 0.010

Accuracy is deliberately not the headline. “Never escalate” scores 80% here.

The threshold is chosen, not defaulted. It is the lowest value whose precision still clears 0.60, because the errors are asymmetric in *both* directions: a missed serious presentation hurts a patient today, and a prompt that fires too often trains clinicians to dismiss the panel — which hurts a patient later. Alarm fatigue is a patient-safety problem.

Deterministic criteria are checked independently and always win. SpO₂ < 92, systolic < 90, focal neurological deficit, chest pain with breathlessness, fever with neck stiffness, bleeding on an anticoagulant, and others. A learned model — especially one trained on synthetic data — must not be able to talk the system out of a hard safety rule. If the artefact is missing entirely, the system degrades to these rather than to nothing.

Deployment. Trained with scikit-learn, exported to JSON, served in pure Python. No ML dependency in the API, no pickle (loading a pickle is arbitrary code execution), and explanations come free from the arithmetic.

The bug the parity gate caught. `CalibratedClassifierCV` fits its calibrator on `decision_function()`, not `predict_proba()`, whenever the base estimator has one. Feeding it the probability produced plausible-looking numbers that did not match the evaluated model at all — an ankle sprain scored 0.23 against a threshold of 0.10 and flagged. The training script now refuses to write the artefact if the pure-Python path diverges from scikit-learn by more than $1e-4$ on the whole test set. Current parity: $4.8e-05$.

The honest framing, which must be said first, not last: the training data is synthetic and the model has never been validated against real outcomes. The metrics demonstrate that the *pipeline* is correct. Real validation needs a labelled retrospective cohort this project does not have.

16. Data analytics: longitudinal statistics

Plainly. Instead of listing old readings, the system tells the doctor whether a vital sign is genuinely trending — and how confident that is.

Why these methods. Clinic series are short (3–12 points), irregularly spaced, and contain transcription errors. That rules out anything assuming normality, even spacing, or large n .

Mann–Kendall for whether a monotonic trend exists. It uses only the *sign* of every pairwise comparison, so a single mistyped reading cannot manufacture a trend, and it assumes no distribution. Ties are corrected; the p-value comes from the normal approximation with a continuity correction, which is conservative at small n — the right direction to err.

Theil–Sen for the slope: the median of all pairwise slopes, ~29% breakdown point, versus least squares which one outlier rotates arbitrarily.

Median absolute deviation for step detection ($|z| \geq 3$ against prior readings, scaled by 1.4826 so the threshold reads in familiar units). With five points, one bad reading inflates the standard deviation enough to hide the very jump you are looking for.

The two tests are complementary, and that is the interesting part. A flat history followed by a sudden jump has *no* monotonic trend — Mann–Kendall correctly says “stable” — and is often the most clinically important pattern. On the seeded demo data:

Ramesh Iyer	bp	n=5	rising	p=0.0275	step=False	+2.13/30d – worth reviewing
Sunita Devi	hr	n=5	stable	p=0.4624	step=True	latest 4.5 robust SDs above prior median

Also computed: recurring complaints counted *per visit* (the same word three times in one consultation is one occurrence), medication starts and stops from prescriptions rather than reported history, and **unresolved** problems — recorded as current, not revisited in 90+ days. That last one is the failure longitudinal memory exists to prevent: a complaint recorded once, never followed up, quietly treated as history because nobody looked.

Everything is labelled with its method and p-value. “BP is rising” and “BP is rising (Mann–Kendall $p=0.03$, $n=6$)” are different statements, and only the second is checkable.

17. Evaluation

Three harnesses, all runnable with `make eval`, all gated in CI.

Extraction and citation grounding

Deterministic — model responses are frozen, so it needs no API key and the number moves when *our* code changes, not when a provider ships a new checkpoint. Scores citation verification (P/R/F1/specificity), field-level extraction P/R/F1 per entity type, and allergy status as a three-way classification.

Red-flag escalation

12 red-flag presentations, 12 benign controls. **12/12 sensitivity, 1/12 false alarms**. The benign set deliberately contains **near-misses** for each hard criterion — abdominal pain without fever, chronic cough without weight loss — because a false-alarm rate measured only on obviously benign cases is not evidence of anything.

This harness was rewritten. The previous version evaluated `kb_ground_red_flags()` — a function that had already been removed from the request path — and required a live database, so it could not run in CI. It kept reporting a recall number for code no consultation ever touched. That is worse than no evaluation, because it produced a metric that looked like evidence.

Risk model

Full metrics in `eval/results/risk_model.json`, regenerated by `make train`. CI retrains and fails if the committed artefact does not reproduce exactly.

These evaluations found real bugs:

- A model answering `"None"` for allergies was parsed as an allergen literally called “None”, which would print in red on the patient’s chart.
- Three red-flag presentations (appendicitis, DKA, TB) were missed — not because the model was miscalibrated but because the *vocabulary to describe them was absent from the lexicon*.

18. Security

Control	What and why
Tenant isolation	RLS in the database, not filters in code
Identity from the token	Never from the request body
Role separation	Only <code>doctor</code> may attest, amend or remove
PostgREST filter injection	Fixed. Filters are a text grammar; the search used to interpolate raw input into <code>or=(name.ilike.*{q}*,...)</code> , so a <code>)</code> or <code>,</code> rewrote the query. Values are now quoted and LIKE wildcards escaped — a bare <code>%</code> used to match every patient in the clinic
Upload limits	Streaming read with a ceiling; content-type allowlist checked before any paid API call
Bounded rate limiting	Keyed by hashed bearer token (behind a proxy an entire clinic shares one IP), LRU-capped so the limiter is not itself a memory-exhaustion vector
Spend ceiling	A daily estimated-USD cap. AI assistance stops; documentation keeps working
CORS	Exact origins, no credentials (bearer auth uses none), explicit method and header lists
No internal error leakage	A global handler returns a correlation id; the detail goes to the log
Secrets	Never in the repo. A CI job fails the build on a tracked <code>.env</code> or a credential-shaped string
Security headers	<code>nosniff</code> , <code>DENY</code> framing, strict referrer, <code>Permissions-Policy</code> allowing only the microphone, <code>noindex</code>
Production config warnings	Startup warns about plaintext origins, disabled rate limiting, an ungated <code>/metrics</code>

PHI-aware logging. Request and response bodies are never logged. Query parameters that could identify a patient are redacted — including `q`, because a log full of `q=Sharma` is a log full of patient names. Metric route labels collapse ids so no patient gets their own metric series. Audit entries record *which fields* changed, not their values, because the audit log is queried and exported and duplicating demographics into it widens the blast radius.

19. Audit trail and attestation

Attestation. A checkbox in the UI, a boolean on the request, and a hard requirement in the database.

`finalize_visit()` raises `check_violation` without it, and raises `insufficient_privilege` if the role may not sign. `attested_by` and `attested_at` are stamped from the caller's identity.

Immutability. A trigger blocks every clinical column on an attested note — including `attested` itself, since un-attesting would be the long way round to rewriting it. Only soft-delete stamps and amendments may change.

Amendments. `amend_visit_note()` appends `{at, by, reason, text}` to the note's `amendments` array and writes an audit entry. Both a reason and the correction text are mandatory. The signed text is never touched — exactly how a paper chart is amended.

The audit log.

- **Append-only by trigger**, so `UPDATE` and `DELETE` are refused for *every* role including `service_role`. Triggers fire regardless of RLS.
- **Hash-chained:** each row carries `prev_hash` and a SHA-256 `row_hash` over its own contents plus its predecessor's hash.
- **`verify_audit_chain()`** walks the chain and returns one row per break, so “tamper-evident” is a query rather than a claim. A test disables the trigger as the table owner, rewrites a row, and asserts the verifier catches it.

A real bug found while testing this. The chain picked its predecessor with `order by at desc, id desc`. `at` defaults to `now()`, which is the *transaction* start time and therefore identical for every row written in one transaction — and `finalize_visit()` writes its audit row inside the clinical transaction. The tie broke on a random UUID, so rows chained in UUID order rather than insertion order. Fixed by chaining on a monotonic sequence.

The honest claim. *Tamper-evident*, not tamper-proof. A database owner can rewrite history. What the chain guarantees is that doing so becomes detectable.

20. Observability

A small metrics registry, deliberately dependency-free, exported in Prometheus text format at `/metrics` (optionally token-gated) and as JSON at `/metrics/summary`.

What is tracked: HTTP requests and latency by templated route; AI calls by capability, provider and outcome; AI latency; **fallback count**; tokens; estimated cost; JSON repairs; citation verdicts; rate-limit rejections; audit write failures; identity cache hit rate; today's estimated spend.

Three health endpoints, deliberately separate:

- `/health` — is the process alive? Touches nothing. An orchestrator that gets a 503 because the database blinked will restart a server that was merely busy.
- `/health/ready` — can it serve? Checks the database, reports which AI capabilities are configured, shows the budget and any production config warnings. A missing AI provider does **not** make the server unready: marking it so would take documentation offline to protect a convenience feature.
- `/metrics` — the series.

Correlation ids. Every request gets one (or reuses an inbound `x-request-id`), it appears in every log line, and it comes back in the response — including in the sanitised 500 body, so a support request can be tied to the exact log line without the client ever seeing the detail.

Honest limitation: in-process only. N workers means N series. Fine for a single-instance prototype; a real deployment scrapes each instance.

21. Testing and CI

426 tests.

Suite	Count	Notes
Backend unit + integration	290	Supabase mocked at the HTTP boundary (respx) — everything between the request and the outbound call is production code
Database	98	Real PostgreSQL, real migrations, no mocks
Frontend	38	Vitest + Testing Library; component tests drive the real consultation page

The database tests build a fresh database from the committed migrations every run — which is itself the test that the migrations reproduce the system. The defect that started this work was a schema that only existed on the live Supabase project: `visits.status` allowed only (`'draft'`, `'approved'`) while the application wrote `'in_progress'`, so every draft save would have failed on a database built from the repository.

CI has no `continue-on-error`. Every gate is one that, if it fails, means something a patient’s record depends on is broken. It also asserts the database tests were not *skipped* — a silent skip would quietly remove the RLS and attestation guarantees from CI.

Plus a hygiene job: no tracked `.env`, no credential-shaped strings, and every evaluation dataset must declare itself synthetic — so a real transcript pasted in during debugging cannot quietly become a committed dataset.

A test-writing principle worth stating in an interview: none of these assert that a function returns something. Each names the failure it prevents. When one breaks, the docstring says why it matters.

22. Built here vs external — and what not to claim

Built here

The consultation workflow. The data model and its provenance rules. RLS policies. `finalize_visit()` and the transactional guarantees. The audit hash chain and its verifier. The provider-failover layer for both LLM and speech. JSON repair. Citation verification. The escalation-risk model, its features, its synthetic cohort, its calibration and its pure-Python inference. The longitudinal statistics. The completeness rubric. Every evaluation harness. The metrics layer. All 426 tests.

Not built here

Differential diagnosis, investigations and treatment recommendations. These come from an external Clinical Synthesis API. What is built is the integration. Claiming the clinical brain would be the single most damaging thing to say in an interview, because one follow-up question exposes it.

Third-party

OpenAI and Sarvam (speech), Gemini and OpenAI (structuring), Supabase (PostgreSQL, auth, PostgREST), scikit-learn (training only).

What must never be claimed

Do not say	Say instead
“It diagnoses patients”	“It surfaces considerations a physician evaluates”
“HIPAA / GDPR / DPDP compliant”	“Designed with DPDP Act 2023 principles in mind; not certified, not audited”
“Clinically validated”	“Evaluated on synthetic data; no clinical validation”
“Tamper-proof audit log”	“Append-only and tamper-evident — a rewrite is detectable”
“Production-ready”	“Working prototype, not deployed”
“I built the clinical decision engine”	“I built the integration; the engine is external”
“99% accurate”	Quote precision, recall and the data they were measured on

23. Interview preparation

30 seconds

Clinical Memory AI turns a spoken doctor–patient consultation into a structured clinical note, and carries that patient’s history across visits as measurable signal rather than as a list. It’s a Next.js frontend, a FastAPI backend, and PostgreSQL on Supabase where multi-tenant isolation is enforced by Row-Level Security rather than by application code. Everything the AI produces is a draft — the database refuses to finalise a note without an explicit physician attestation.

60 seconds

Add: The interesting engineering is in not trusting the AI. Every field the model auto-fills carries the verbatim words from the transcript that produced it, and the server verifies each quote actually appears there before showing it — measured at precision 1.0 on a labelled set, and gated at 1.0 in CI.

There's also an interpretable ML layer: a calibrated logistic regression over 46 named clinical features that answers one question — is this case worth a second look? ROC-AUC 0.90, recall 0.82, and calibration error down from 0.10 to 0.01. It's trained on synthetic data, so I present it as unvalidated, which is stated in the API response and printed in the UI.

And the longitudinal memory does real statistics: Mann–Kendall trend tests and robust step detection, so it can say “this patient's BP is genuinely rising, $p=0.03$ ” rather than just listing old readings.

2 minutes

Add:

The problem. A doctor has ten minutes to listen, examine, recall history, decide on tests, prescribe — and type. The typing loses.

The architecture constraint that shaped everything. A clinical note is a legal document, and AI output is unreliable. So the system is built so AI output is always a *proposal* and only a physician's signature makes it a *fact*. That's why the rules live in Postgres rather than in the API: signing is one function, in one transaction, that requires attestation, requires the doctor role, refuses to re-sign a signed visit, and rejects a stale version. Those hold even if my API code is wrong.

What I'd highlight. Isolation is RLS — no query in my codebase filters by clinic, and 31 database tests actively try to cross the boundary. The audit log is append-only by trigger and hash-chained, with a `verify_audit_chain()` function so tamper-evidence is a query, not a claim. Provider failover is real and cross-vendor, with the fallback rate as a metric.

What I'd be honest about. The differential diagnosis engine is an external API — I built the integration, not the clinical brain. The risk model is trained on synthetic data. It isn't deployed. Rate limiting is per-process.

5 minutes (technical)

Cover, in this order:

1. **Problem and constraint** (as above).
2. **Isolation.** RLS, `current_clinic_id()`, `USING` vs `WITH CHECK`, why not application filters. Mention the missing `soap_notes` UPDATE policy bug — PostgreSQL returns 204 for zero rows, so finalising a draft reported success and wrote nothing. Silent clinical data loss that looked like success. Now there's a structural test asserting every writable clinical table has all three policies.
3. **Transactional finalize.** Five REST calls → one function. Attestation, role, immutability, optimistic locking, all in the database. Two connections racing in a test.
4. **AI reliability.** Ordered chains, cross-vendor, transient vs non-transient handling, structure-only JSON repair, and the rule that a 200 carrying prose is a failure.

5. **Citation verification.** The exact/near/unsupported ladder and why the content-word requirement is the safety property. Numbers.
6. **The ML.** Why logistic regression over GBT (I trained the GBT, it was marginally better, I didn't deploy it). Calibration and why class weighting makes it necessary. The threshold choice and alarm fatigue. The parity gate and the `decision_function` bug it caught.
7. **Statistics.** Mann–Kendall and Theil–Sen and why not least squares. The complementary step detector.
8. **Evaluation as regression testing**, and the two real bugs it found.
9. **Honest limitations.**

Closing line to have ready: *“The thing I’d most want to be judged on isn’t any single feature — it’s that every claim in the README has a test or a number behind it, and the ones that don’t have been removed.”*

24. Interview questions and answers

For each: what the interviewer is testing · a natural answer · the likely follow-up · the one thing to remember.

Python and FastAPI

Q1. Why FastAPI over Django or Flask? *Testing:* whether you chose or defaulted. **A:** Two reasons specific to this system. The payloads are deeply nested and come from a language model, which makes them untrusted input — Pydantic validates them at the boundary rather than in scattered `if` statements. And the workload is I/O-bound: one consultation fires speech, LLM and decision-support calls, and `asyncio.gather` runs the three decision-support lanes in parallel. Django’s ORM would also be dead weight, because I go through PostgREST to keep RLS in the loop. *Follow-up:* “What would you lose without an ORM?” → Migrations and query composition; I write raw SQL migrations, which is fine here because the schema is small and the policies have to be hand-written anyway. *Remember:* `async` + validation of untrusted model output.

Q2. Explain how you handle `async` in the request path. **A:** Every route is `async def`, all outbound HTTP is `httpx.AsyncClient`, and independent calls are gathered. One pooled client for the process, not one per request — a fresh connection pool per call meant a TCP and TLS handshake on every database read, which was the largest avoidable latency in the path. *Follow-up:* “What if you had a blocking call?” → `run_in_threadpool`, or it blocks the event loop for every concurrent request. *Remember:* the pooled-client fix is a concrete, measurable decision.

Q3. How do you keep configuration safe and correct? **A:** A Pydantic `Settings` object where the required fields have no defaults, so constructing it fails and the server refuses to start. There’s a validator on `ENVIRONMENT`, and a `production_warnings()` method that flags plaintext origins, disabled rate limiting and an ungated `/metrics` at startup. A test asserts `.env.example` documents every setting and ships no secret values, so the template cannot drift. *Follow-up:* “Why fail at startup rather than on first request?” → Because the first request is a patient’s, and a 500 mid-consultation is a much worse place to discover a missing key. *Remember:* fail fast, and test that the template stays in sync.

Q4. How do you test code that calls external APIs? A: `respx` mocks at the HTTP boundary, so everything between the request and the outbound call is production code — middleware, dependencies, validation, error translation. I don't mock my own functions. That's how the failover matrix gets tested: 503 falls through, 401 skips the vendor, a 200 with prose counts as a failure. *Follow-up*: "What about the database?" → Not mocked. Those run against real PostgreSQL with the real migrations, because mocking RLS would test the mock. *Remember*: mock the third party, never your own layer.

Next.js and TypeScript

Q5. Why are your pages client components? A: They need the microphone, `AudioContext`, intervals and live state. Server components would buy nothing since everything is behind auth and personalised. The trade-off is no server-side session check, which is why the backend re-derives identity from the token on every request — the client checks only shape the UI. *Follow-up*: "So a user could hit a page while logged out?" → Yes, and they'd see a loading state, then a redirect, and every API call would 401. No data leaks, because authorisation isn't in the client. *Remember*: the client checks are UX; the backend is the authority.

Q6. Your build was failing on a clean checkout. What happened? A: `createClient` was called at module scope with `process.env...!`. Next still evaluates client-component modules during prerender, so `next build` threw "supabaseUrl is required" on any checkout without `.env.local` — including CI. I made the client lazy behind a Proxy, so construction happens on first property access. Building a project shouldn't require production credentials, and now CI builds with none deliberately. *Follow-up*: "Why a Proxy rather than changing the call sites?" → Every page already imported `supabase` and called `supabase.auth.*`; a Proxy kept the ergonomics and deferred construction with a clear error if unconfigured. *Remember*: a real reproducibility bug with a small, clean fix.

Q7. How do you handle API errors in the UI? A: One `errorMessage(response)` helper. The backend distinguishes 409 from 403 from 429 deliberately, and the UI used to collapse them into "Save failed (409)" — which is exactly the moment the distinction matters, because one means reload, one means ask a colleague, one means wait. It also never shows a raw non-JSON error body, so a database error string can't reach the screen. All of it is unit-tested. *Remember*: error translation is a product feature, not plumbing.

Q8. What do your frontend tests actually cover? A: The flows with clinical weight: Sign & save is disabled until the attestation box is ticked; the signed confirmation appears only after a successful save; a 409 produces an actionable message and does *not* show success; a resumed draft carries its version back so the backend can detect a clobber; and the escalation panel shows its reasons, their weights, and the model's stated performance. Plus the prescription safety checks, where every test is either a conflict that must fire or a false positive that must not. *Follow-up*: "Why not E2E with Playwright?" → It's the honest gap. Component tests with a fake backend catch the logic; they don't catch a broken deploy. *Remember*: name the gap before they do.

PostgreSQL, Supabase, RLS

Q9. Explain Row-Level Security to someone who has not used it. A: A `WHERE` clause the database enforces on every query, no matter who wrote the query. Here every clinical table has policies restricting rows to `clinic_id = current_clinic_id()`, where that function resolves the caller's clinic from their JWT. The backend forwards the user's own token to PostgREST, so the database evaluates the rule against

their identity. *Follow-up*: “Why not filter in application code?” → Because that fails the day someone forgets one clause on one query. Here no query filters by clinic at all. *Remember*: isolation that doesn’t depend on anyone remembering anything.

Q10. What’s the difference between USING and WITH CHECK? *Testing*: whether you actually wrote policies or copied them. **A**: USING decides which rows you can see and target. WITH CHECK validates rows you’re writing. Without WITH CHECK on an UPDATE, you could take a row you legitimately own and set its clinic_id to another clinic — walking it out of the tenant. I have a test that tries exactly that and expects InsufficientPrivilege. *Remember*: WITH CHECK is what stops a row leaving its tenant.

Q11. Tell me about a bug you found in your own RLS setup. **A**: soap_notes had RLS enabled with SELECT, INSERT and DELETE policies but no UPDATE policy. PostgREST returns 204 for an update matching zero rows, so finalising a draft reported success and wrote nothing — the note was silently lost. It only worked in production because the live schema had drifted from the committed migrations. I added the policy, and a structural test asserting every writable clinical table has all three commands, so the class of bug can’t recur. *Follow-up*: “How did you find it?” → Building the database from the committed migrations in CI. It had never been done. *Remember*: the silent 204 is the detail that makes this a good story.

Q12. How do you know your migrations actually work? **A**: Every database test run creates a fresh database and applies every migration from scratch. That’s the test. It caught visits.status allowing only ('draft', 'approved') while the application wrote 'in_progress' — every draft save would have failed on a database built from the repo. *Remember*: “the migrations are the schema” is only true if you prove it.

Q13. Why SECURITY INVOKER on finalize_visit()? **A**: So RLS still applies. SECURITY DEFINER would run as the owner and bypass every policy, turning the function into a tenant-isolation bypass — call it with another clinic’s patient_id and it’d work. As invoker, that patient simply doesn’t exist for the caller, and there’s a test asserting NoDataFound. *Follow-up*: “But you do use SECURITY DEFINER somewhere.” → Twice, both minimal: current_clinic_id() reads only the caller’s own row, and write_audit() stamps actor and clinic from the caller’s JWT rather than accepting them as arguments — otherwise any user could forge an entry as anyone. *Remember*: know exactly where you used the dangerous option and why it’s safe.

Q14. What does the service_role key do and where do you use it? **A**: It bypasses RLS entirely. Exactly one use: resolving auth_uid → users during authentication, because the caller can’t read users until we know their clinic. Every other use would silently discard tenant isolation, so it’s commented as a review flag. *Remember*: one use, named, justified.

Multi-tenancy and architecture

Q15. How would this scale to 500 clinics? **A**: RLS scales fine — it’s an indexed predicate, and the clinic-scoped indexes are there. The parts that don’t: rate limiting and the spend cap are per-process, so N workers means N× the limit; both need Redis. Metrics are per-process and need per-instance scraping. And /consultations caps at 500 rows with no pagination, which would need cursor pagination. *Follow-up*: “Would you shard per clinic?” → Not at 500. The isolation is already at row level and the data volume is small; sharding would add operational complexity for no benefit. *Remember*: name the specific things that break, not “it’d need work”.

Q16. Walk me through what happens when a doctor signs a note. A: (Section 9 of this guide — go through the ten steps of `finalize_visit()` and stress that it's one transaction.) Follow-up: “What if the audit write fails?” → It's inside the transaction, so the whole thing rolls back. That's deliberate: a clinical write with no audit entry is worse than a failed write the doctor can retry. Remember: the audit is *inside* the transaction.

Q17. How do you handle two doctors editing the same consultation? A: Optimistic locking. `GET /visits/{id}` returns `version`; the client sends it back; `finalize_visit()` does `SELECT ... FOR UPDATE` then compares. The second writer blocks on the row lock, then fails with `serialization_failure`, which maps to 409 and a “reload and try again” message. Tested with two real connections racing on a committed draft. Follow-up: “Why optimistic rather than pessimistic?” → A consultation lasts ten minutes. Holding a lock for that long blocks legitimate work and breaks if the browser closes. Conflicts are rare; detecting them is enough. Remember: “two clinicians, last-write-wins, no signal to either” is the failure it prevents.

AI pipelines, LLMs, structured extraction

Q18. How do you handle an LLM returning malformed JSON? A: Three layers. Ask for JSON mode. Repair the structure if it's truncated — close what was left open, drop a dangling key. And if it still won't parse, treat it as a *failure* and fall over to the next provider, because a 200 carrying prose used to produce an empty note with no error anywhere. The repair rule is: structure yes, content never. A truncated field is dropped, not guessed. Follow-up: “How do you know the repair is safe?” → A property test truncates a real note at every offset and asserts the result is always a subset of the original with unchanged values. Remember: “repair structure, never invent content.”

Q19. Describe your provider failover. A: An ordered `(provider, model)` chain: all Gemini models, then OpenAI. Models within a vendor first because model overload is the common failure; then a different vendor, because a fallback sharing an outage with its primary isn't a fallback. Transient errors advance to the next candidate; 401 or 400 skips the rest of that vendor's models — a bad key won't get better — but still falls through to the next vendor. Fallbacks are counted as a metric. Follow-up: “Why not retry the same model?” → I do implicitly via the short backoff between candidates, but retrying an overloaded model is usually slower than trying a different one, and this sits in the request path of a live consultation. Remember: the 401-skips-the-vendor rule is the detail that shows thought.

Q20. How do you stop the model hallucinating clinical facts? A: Four things. The prompt forbids inventing anything and tells the model evidence will be checked. Every auto-filled field must carry a verbatim quote. The server verifies each quote against the transcript and drops what it can't find. And nothing becomes a confirmed clinical fact until a physician attests — drafts write no facts at all. Follow-up: “Can it still hallucinate the *note text*?” → Yes. Verification covers evidence, not prose. That's why the physician reviews, and why the raw transcript is shown alongside the note. Remember: be clear about what verification does *not* cover.

Q21. Your prompts are long. How do you manage them? A: They're in one module, `ai/prompts.py`, because they're the actual clinical-safety surface — every constraint that stops the model diagnosing or inventing lives there. Keeping them in one place makes them reviewable in a pull request and quotable in documentation. Follow-up: “How do you know a prompt change didn't break anything?” → That's the

honest gap: the deterministic evaluation freezes model responses, so it tests my pipeline, not the prompt. `-live` runs the same cases through the real provider, but it isn't in CI because it costs money and isn't deterministic. *Remember:* know which half your evaluation covers.

Q22. Why not fine-tune a model? **A:** No labelled data, and it wouldn't address the actual failure modes — which are hallucinated evidence and unreliable providers, not the base model being bad at English clinical vocabulary. Verification and failover fix real problems; fine-tuning would fix an imagined one. *Remember:* answer with the failure modes you measured.

Speech-to-text

Q23. How do you handle Hindi-English code-mixing? **A:** By *not* pinning a language. Pin to Hindi and the English clinical terms get mangled — and those are the words that matter. Pin to English and the Hindi disappears. Both providers are asked to auto-detect. OpenAI `gpt-4o-transcribe` is preferred for code-mixed audio, Sarvam is the fallback and is built for Indian languages. *Follow-up:* “How do you measure transcription quality?” → I don't, and that's a real gap: it'd need a labelled audio set with reference transcripts, which means recording real consultations. The mitigation is showing the physician the raw transcript next to the note. *Remember:* the language-pinning explanation shows domain understanding.

Q24. Why chunk the audio instead of streaming? **A:** The providers here are batch transcription endpoints, not streaming ones. So the browser encodes 16-bit mono WAV and posts a slice every ~12 seconds, sending only new audio. Twelve seconds is a compromise: shorter costs more, longer makes the note visibly lag the conversation. *Follow-up:* “What breaks if a chunk fails?” → That slice's speech is lost. Failover reduces it; there's no client-side retry buffer, which is a known gap. *Remember:* know the trade-off in the chunk size.

ML, features, evaluation

Q25. Walk me through your ML model. **A:** (Section 15.) An L2 logistic regression over 46 named clinical features, class-weighted, isotonically calibrated, answering only “is this worth a second look”. ROC-AUC 0.90, recall 0.82 at precision 0.66, calibration error 0.10 → 0.01. Trained on a synthetic cohort, so it's unvalidated, and the API says so. *Remember:* lead with what it answers, not with the algorithm.

Q26. Why logistic regression and not something stronger? **A:** I trained a gradient-boosted tree as a reference. It was marginally better — ROC-AUC 0.909 vs 0.902. I didn't deploy it, because the requirement is that a physician can read why a case was raised and disagree with it. In a linear model the contribution of a feature is coefficient × value, so the explanation falls out of the arithmetic with no separate explainer that could disagree with the model. Two points of AUC on synthetic data doesn't buy that. *Follow-up:* “What about SHAP on the tree?” → SHAP is an approximation of the model. For a clinical prompt I'd rather have an explanation that's exactly the model, and I'd have to explain to a clinician what a Shapley value is. *Remember:* you *measured* the alternative. That's what makes it a decision.

Q27. Your data is synthetic. Isn't the evaluation meaningless? *Testing:* intellectual honesty. This is the question to want. **A:** It's meaningless as a clinical claim, and I say so first — in the model card, in the API response, and in the UI. What it does establish is that the pipeline is correct: features extract as intended, the model recovers a signal it can't see directly, calibration works, and the serving path matches the

training path. It's not circular, because the labels come from a rule that's deliberately non-linear — a threshold on the *count* of deranged vital systems, an age $\times$ severity interaction, a comorbidity multiplier that only applies to cardiorespiratory presentations, plus 4% label noise. A logistic regression can't recover that exactly. If I'd generated labels linearly, the metrics would measure nothing. *Follow-up*: "What would real validation need?" → A labelled retrospective cohort of real consultations with recorded outcomes — did this patient get admitted, escalated, come back within 48 hours — plus ethics approval and a prospective silent-mode evaluation before it influenced anyone's care. *Remember*: say "it establishes the pipeline, not the clinical validity" and mean it.

Q28. Explain precision, recall and F1 in this system's terms. **A:** Precision: of the cases we flagged, how many warranted a second look — 0.66, so two in three prompts are worth acting on. Recall: of the cases that warranted one, how many we caught — 0.82. F1 is their harmonic mean, 0.73. I report PR-AUC rather than ROC-AUC as the headline for the imbalanced view, because with a 20% positive rate the baseline PR-AUC is 0.20 and mine is 0.79 — ROC-AUC flatters imbalanced problems. *Follow-up*: "Why not accuracy?" → "Never escalate" scores 80% here. Accuracy is the metric that hides the failure. *Remember*: always state the baseline next to the number.

Q29. What is calibration and why did you need it? **A:** A calibrated model's "0.7" means it happens seven times in ten. `class_weight="balanced"` deliberately distorts the intercept to trade precision for recall, so the raw output isn't a probability at all. Since the number is shown to a clinician next to a threshold, it has to mean what it says. I fit an isotonic calibrator on a held-out slice that never touched the coefficients; expected calibration error went from 0.100 to 0.010, Brier from 0.076 to 0.063. *Follow-up*: "Isotonic or Platt?" → Isotonic, because it's non-parametric and I had no reason to assume the miscalibration was sigmoidal. It needs more data and can overfit on a small calibration set — 1,125 points is enough here. *Remember*: "class weighting is why the raw output isn't a probability."

Q30. How do you choose an operating threshold? **A:** Not 0.5. It's the lowest threshold whose precision still clears 0.60, because the errors are asymmetric in *both* directions. A missed serious presentation hurts a patient today. But a prompt that fires too often trains clinicians to dismiss the panel — and then they dismiss the one that mattered. Alarm fatigue is a patient-safety problem, not a UX complaint, so I gated the false-alarm rate too. *Follow-up*: "How would you choose it with real data?" → From the actual cost ratio: what does a missed escalation cost versus a redundant review, in the clinic's terms, and pick the threshold minimising expected cost. *Remember*: the alarm-fatigue argument is the one that shows domain thinking.

Q31. Tell me about a bug in your ML code. **A:** The best one. `CalibratedClassifierCV` fits its calibrator on `decision_function()` — log-odds — not `predict_proba()`, whenever the base estimator exposes one. My hand-written inference fed it the sigmoid output. The probabilities looked plausible: an ankle sprain scored 0.23 against a threshold of 0.10 and flagged. Every benign case flagged. I caught it because I'd written a parity gate: the training script runs my pure-Python inference over the whole test set and refuses to write the artefact if it diverges from scikit-learn by more than $1e-4$. It's now $4.8e-05$. *Follow-up*: "Why hand-write inference at all?" → So the API has no scikit-learn dependency and the artefact is JSON rather than a pickle — loading a pickle is arbitrary code execution, which isn't a property a clinical system should ship with. The parity gate is the price of that. *Remember*: this answer shows you build the check *before* you need it.

Q32. Why 46 hand-built features instead of embeddings? A: Three reasons. Explainability — “the vector was close to the escalation cluster” isn’t something a clinician can act on. Auditability — I can read the learned coefficients against clinical expectation, and they order correctly: focal neurological deficit +4.63, bleeding +4.00, altered mental state +3.34. And stability — a vocabulary shift moves an embedding; it doesn’t move “SpO₂ = 88”. *Follow-up*: “Would embeddings score better?” → Almost certainly. And I couldn’t ship them, because the failure mode of an unexplainable clinical prompt is that clinicians ignore it. *Remember*: the coefficient ordering is a check you can only run on named features.

Q33. How does the model handle missing data? A: Explicitly. Roughly a third of real encounters record no vitals, so the synthetic cohort reproduces that, and `vitals_absent` is itself a feature — an encounter with no objective anchor is genuinely a reason to look again. Absent vitals give 0 for the threshold features rather than an imputed value, because imputing a normal blood pressure for a patient nobody measured is a lie the model would then believe. *Remember*: “training on complete data is how a demo model degrades silently in the field.”

Q34. What is class imbalance and how did you handle it? A: 20% positive. `class_weight="balanced"` reweights the loss so the minority class isn’t ignored; then calibration corrects the probabilities that reweighting distorts. And I report PR-AUC with its baseline rather than accuracy. *Follow-up*: “Why not SMOTE?” → Synthesising minority examples in a feature space of binary clinical flags produces combinations that aren’t clinically coherent. Reweighting doesn’t invent patients. *Remember*: the “SMOTE invents incoherent patients” point.

Statistics and analytics

Q35. Why Mann–Kendall instead of linear regression? A: Three points to twelve, irregularly spaced, with occasional transcription errors. Least squares assumes normal residuals and is rotated by a single outlier — a mistyped “210” for a heart rate would produce a significant trend. Mann–Kendall uses only the sign of each pairwise comparison, so it’s robust to that and assumes no distribution. There’s a test with exactly that outlier asserting no trend is reported. *Follow-up*: “What’s the cost?” → Less power than least squares when the data really is well-behaved and normal. For $n=5$ with a possible typo, robustness is worth more. *Remember*: name what you gave up.

Q36. What is Theil–Sen and why use it? A: The median of all pairwise slopes. It answers “how fast”, where Mann–Kendall answers only “is there a trend”. Its breakdown point is about 29%, so roughly a third of the readings can be corrupted before the estimate goes wrong — versus least squares, where one point can do it. *Remember*: Mann–Kendall for *whether*, Theil–Sen for *how fast*.

Q37. Why do you have both a trend test and a step detector? A: They catch complementary patterns, and the second one is the clinically interesting case. A patient whose heart rate is 72, 75, 71, 74, 73, then 120 has *no* monotonic trend — Mann–Kendall correctly says stable — but that jump is exactly what a doctor needs to see. The step detector uses median absolute deviation rather than standard deviation, because with five points one bad reading inflates the SD enough to hide the very jump you’re looking for. *Remember*: the worked example. It makes the point instantly.

Q38. How do you avoid crying wolf on trends? A: A minimum of three points before any claim is made, alpha at 0.10 rather than 0.05 because these series are short and I’d rather flag for review than miss, robust methods so one bad reading can’t create a trend, and every claim labelled with its test and p-value. “BP is

rising” and “BP is rising (Mann–Kendall $p=0.03$, $n=6$)” are different statements, and only the second is checkable. *Remember*: alpha 0.10 is a choice with a reason, not a slip.

Retrieval and RAG

Q39. Is there RAG in this system? **A:** No vector store and no embedding retrieval, and that was a decision rather than an omission. There are two retrieval problems here. Patient history is a bounded set of rows for one patient, ordered by time — SQL is exactly the right tool, and it’s also the only tool that respects RLS. Clinical knowledge is owned by the external decision-support service. A vector database would have been a keyword on a CV. *Follow-up*: “When would you add one?” → When there’s a corpus too large to scan and too unstructured to index — clinical guideline PDFs, say. Then I’d need chunking, a retrieval evaluation with precision@k, and grounding checks. I wouldn’t add the infrastructure before the corpus. *Remember*: this answer is stronger than pretending to have RAG.

Q40. What grounding do you have, then? **A:** Citation verification, which is the part that actually reduces hallucination risk. Every field the model fills carries a verbatim quote, the server checks it against the transcript, and unverifiable quotes are dropped. It’s measured — precision 1.0, recall 1.0 on a labelled set — and gated in CI. Plus `evidence_coverage_pct` per extraction, so “how much of this note is backed by something the microphone heard” is a number. *Remember*: grounding against the transcript, not against a corpus.

Security

Q41. Walk me through a security issue you found and fixed. **A:** PostgREST filter injection in patient search. The query was interpolated into `or=(name.ilike.*{q}*,phone.ilike.*{q}*,uhid.ilike.*{q}*)`. PostgREST filters are a text grammar, not bound parameters, so a `)` or `,` closed the group early and the rest was parsed as further filter clauses. RLS contained the blast radius to the caller’s own clinic, but the query the database ran wasn’t the one I wrote — which is the definition of injection. The fix quotes values per PostgREST’s escaping rules and neutralises LIKE wildcards, because a bare `%` matched every patient in the clinic. There’s a test with a real parser that asserts the payload stays inside one quoted value. *Follow-up*: “How bad was it really?” → Contained by RLS, so not a cross-tenant breach. Still a real bug, and I’d rather explain the class than argue about severity. *Remember*: explain *why* it’s injection even though RLS contained it.

Q42. How do you keep PHI out of your logs? **A:** Request and response bodies are never logged — transcripts and notes are all PHI and a log aggregator isn’t a clinical record system. Query parameters are redacted against a deliberately broad list including `q`, because a log full of `q=Sharma` is a log full of patient names. Metric route labels collapse ids so no patient gets their own metric series. Audit entries record which *fields* changed, not their values. *Follow-up*: “What about errors?” → A global handler returns a correlation id and a generic message; the detail goes to the log only. There’s a test that raises an exception containing a connection string and asserts it doesn’t appear in the response. *Remember*: the `q=Sharma` line lands every time.

Q43. Why is your rate limiter keyed on a hashed token? **A:** It was keyed on IP. Behind a proxy an entire clinic shares one address, so one busy consultation could rate-limit the whole building. Authenticated requests are now keyed by a SHA-256 hash of the bearer token — a stable identifier, not the credential

itself — with IP as the fallback for unauthenticated calls. The bucket store is also LRU-capped now, because the unbounded dictionary meant rotating source addresses turned the limiter itself into a memory-exhaustion vector. *Follow-up*: “It’s per-process.” → Yes, and I say so. N workers means $N \times$ the limit. It’s protection against a runaway client and accidental cost, not a distributed attacker. Correct needs Redis. *Remember*: two distinct bugs — the key and the unbounded store.

Q44. How do you control AI spend? **A:** A daily estimated-USD ceiling. Rate limiting caps requests per minute, which isn’t the same thing — a cheap call and an expensive one cost the same against that limit. When the budget is exhausted, the AI routes return 429 with a clear message and **documentation keeps working**. That last part is the design decision: losing AI assistance must not stop a physician recording a patient. There’s a test asserting the local risk endpoint still works when the budget is blown. *Remember*: “the cheap call and the expensive call cost the same rate-limit token.”

Healthcare AI and safety

Q45. How do you stop the AI from making clinical decisions? **A:** Structurally, not by policy. Four states are kept distinct: extracted facts, AI suggestions, physician-confirmed facts, and externally sourced evidence. A draft writes **zero** clinical facts, so an unsigned note never becomes memory. Finalising requires attestation and the doctor role, enforced in the database. And decision support is always labelled physician-review-only and sourced. *Follow-up*: “What if the physician just clicks through?” → Then they’ve signed it, and the audit log records who and when. The system can’t make a clinician careful; it can make sure there’s always a named human who accepted each fact, and that ignored safety warnings are recorded with the reason given. *Remember*: “a draft writes zero facts” is the crispest version.

Q46. What is alarm fatigue and how does your design address it? **A:** When a system warns too often, clinicians learn to dismiss the warnings — including the one that mattered. It’s a documented cause of patient harm in hospital monitoring. Three things: the red-flag panel is capped at four items because beyond that people skim past the whole panel; the risk threshold is chosen for precision ≥ 0.60 rather than maximum recall; and the false-alarm rate is gated in CI against benign controls that include deliberate near-misses. The single false alarm in my results — a 48-year-old smoker with a three-week cough — I left in rather than tuning away, because that patient arguably *is* worth a second look and moving the threshold to prettify the number would be fitting to the evaluation set. *Remember*: leaving the false alarm in is the credible detail.

Q47. What would you need before this touched a real patient? **A:** Clinical validation on a real labelled cohort with ethics approval. A prospective silent-mode evaluation where it runs but influences nothing, so its real-world error rate is measured before anyone acts on it. A DPDP compliance review and a data-protection impact assessment. Penetration testing. A deployed, monitored, backed-up environment with tested restores. And an incident process for when it gets something wrong, because it will. *Remember*: “silent mode first” is the answer that shows you understand clinical software rollout.

Q48. What’s the most dangerous failure mode of this system? **A:** Silent wrongness. Not a crash — a crash is visible. The dangerous case is a plausible-looking note with a fabricated detail that a rushed physician signs. That’s why citation verification exists, why the raw transcript is shown next to the note, and why the highest-priority CI gate is that citation precision must be exactly 1.00. Second is the inverse: a doctor reading an empty decision-support panel as reassurance. Which is why “unavailable” is shown explicitly rather than as emptiness. *Remember*: both directions. Most people only name the first.

System design and failure handling

Q49. What happens when every AI provider is down? A: Speech returns 502 with “your recording was not lost — press record again or type manually”. Extraction and the live lane return empty with `available: false`. Decision support fails open with a banner. The escalation prompt keeps working because it’s local — no network, no cost. And the entire documentation workflow, including signing, is unaffected. Each of those is a test. *Follow-up:* “Why does the risk endpoint keep working?” → Because it’s a dot product and a table lookup in-process. That’s a direct benefit of exporting the model to JSON instead of calling a model-serving endpoint. *Remember:* the degradation ladder, in order.

Q50. If you had another week, what would you do? A: In order. First, containerise the backend and actually deploy it — the Cloudflare path is configured and unverified, which is the biggest gap between “works on my machine” and “works”. Second, Redis for rate limiting and the spend cap, so they’re correct across instances. Third, a real end-to-end test against a deployed instance, because component tests with a fake backend don’t catch a broken deploy. Fourth, code the diagnoses — differentials carry ICD-10 from the external service but what gets saved is free text, which makes the longitudinal analytics weaker than they could be. What I wouldn’t do is add features. The gap isn’t capability, it’s that nothing is deployed and nothing is validated. *Remember:* ordered, specific, and ends by declining to add features.

Last updated: 2026-09-11. Every number in this guide comes from `backend/eval/results/`, regenerated by `make eval` and `make train`.